KLAi

# Matrices: tables of numbers that also move vectors, and the core arithmetic of a neural-network layer

*A language model has billions of parameters. What are those numbers, and what does the model do with them?*

Mathematics for AI and Machine Learning | Session 3 of 18 | Linear algebra
KLAi Professional Learning | Jotham Martin Wambi

# Today in one view: what you will leave with

## By the end of today you can



Read a matrix as a table and as a transformation, and state its shape (rows by columns)

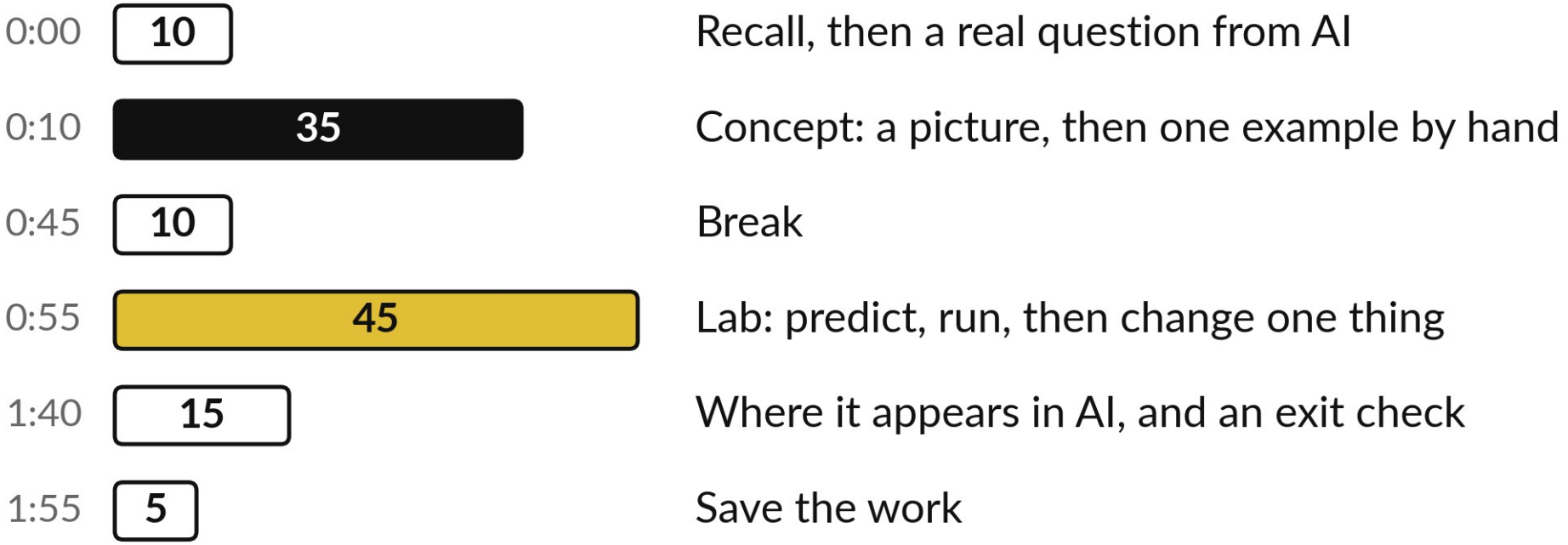


Multiply a matrix by a vector and a matrix by a matrix by hand, and say when the shapes allow it



Compute a layer y = W x + b, and count the parameters of a small network, in Python and in a spreadsheet

# Your two hours: matrices, then the lab, then parameters




Pictures and hand calculations first. Then the lab on products, layers and parameters.

# Three questions from last time

1. What is the dot product of (1, 2, 3) and (4, 0, 2)? Is the result a vector or a number?

2. What is the L2 norm (length) of the vector (2, 3, 6)?

3. Why does a search tool rank by cosine similarity and not by the raw dot product?

Answer aloud first. Then we check together.

# Answers to the three questions from Session 2

| Question | Short answer |
|---|---|
| **What is the dot product of (1, 2, 3) and (4, 0, 2)? Is the result a vector or a number?** | 10. It is one number: $1 \times 4 + 2 \times 0 + 3 \times 2$. |
| **What is the L2 norm (length) of the vector (2, 3, 6)?** | 7, because 4 + 9 + 36 = 49 and the square root of 49 is 7. |
| **Why does a search tool rank by cosine similarity and not by the raw dot product?** | Cosine similarity divides by both lengths, so a long vector does not win just for being long. Only the direction counts. |

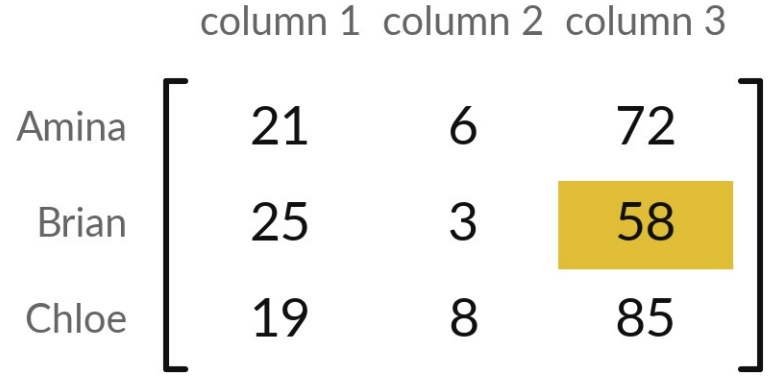
If one answer surprised you, look again at the Session 2 slides before the lab.

# Words we use today

| | Word | What it means |
|---|---|---|
|  | **Matrix** | A table of numbers with m rows and n columns. Its shape is m × n. |
|  | **Matrix multiplication** | A rule that combines two matrices. Each entry of the answer is a dot product of a row and a column. |
|  | **Transpose** | The matrix flipped so that rows become columns. It is written with a small T. |
|  | **Identity matrix** | A square matrix of 1s on the diagonal and 0s elsewhere. It leaves every vector unchanged. |
|  | **Parameters** | The numbers inside a model that training sets: its weights W and biases b. |

# A matrix is a table of numbers with rows and columns

**L is a 3 × 3 matrix: 3 rows and 3 columns**




**Matrix: a table of numbers. Here L holds three learners from Session 2, one per row: age, hours, score.**

Each row is a vector: Amina = (21, 6, 72).

Gold entry: row 2, column 3 is 58, Brian's score.

Illustrative (made-up numbers to show the arithmetic).

# A capital letter names the matrix, and two small numbers pick one entry

Shape $A \in \mathbb{R}^{m \times n}$ (A is m × n)

Entry in row i, column j $A_{ij}$

Our table $L \in \mathbb{R}^{3 \times 3}, \qquad L_{23} = 58$

| Symbol | What it means | In our data |
|---|---|---|
| **A, L** | A matrix is written with a capital letter. A vector is a lowercase bold letter. | L = the learner table |
| **m × n** | The shape: m rows by n columns. Rows are always named first. | 3 × 3 |
| **∈ and ℝ** | ∈ reads "is a". ℝ is the real numbers: ordinary numbers with decimals. | L is a 3 × 3 table of real numbers |
| **$A_{ij}$** | The entry in row i, column j of A (small i and j below the A). | $L_{23}$ = 58 |

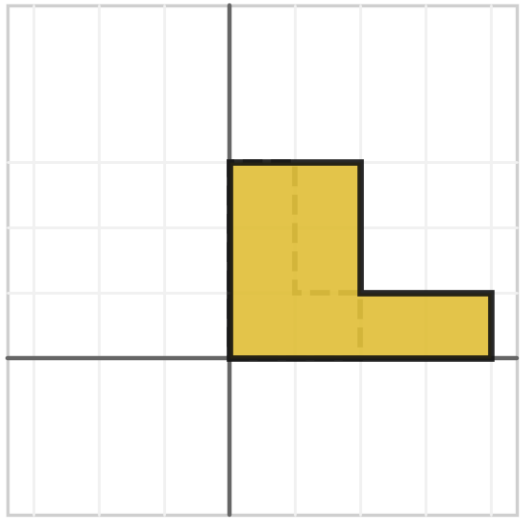
Python counts from 0, so $L_{23}$ has row index 1 and column index 2. A spreadsheet counts from 1: INDEX(range, 2, 3).

# A matrix can also move points: scaling, rotating and shearing a shape

**Scale**

rows (2, 0) and (0, 1)

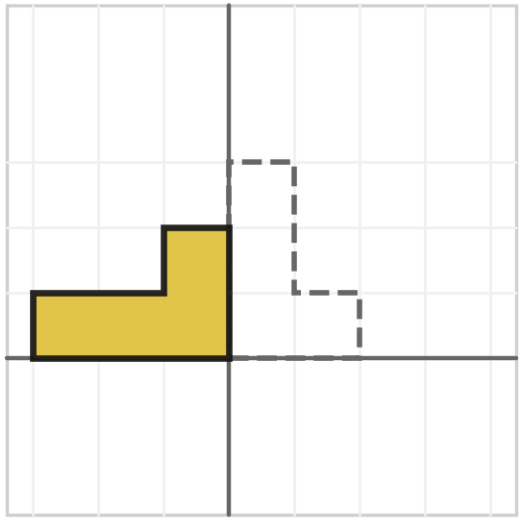
doubles every x, keeps y



**Rotate**

rows (0, −1) and (1, 0)

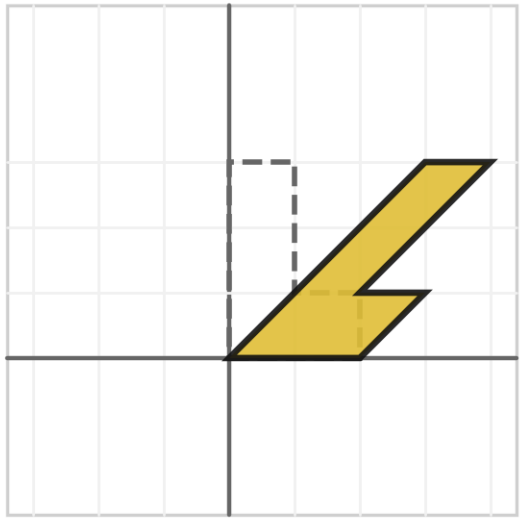
quarter turn anticlockwise



**Shear**

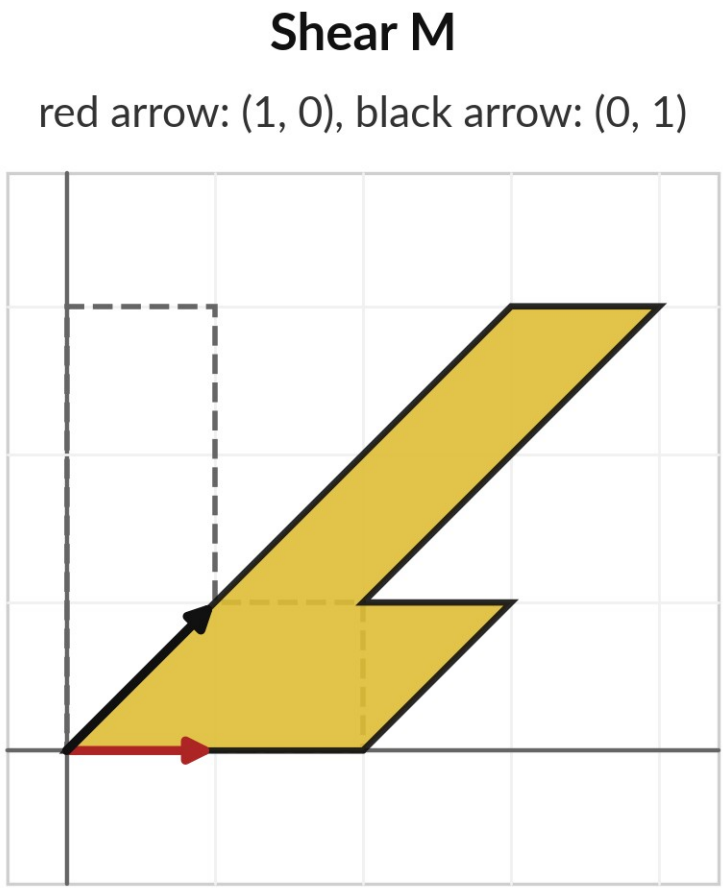
rows (1, 1) and (0, 1)

slides each point right by its height



Grey dashed: the original shape. Gold: the result. The rotation sends the corner (1, 3) to (−3, 1).

# The columns of a matrix show where the two axis arrows land



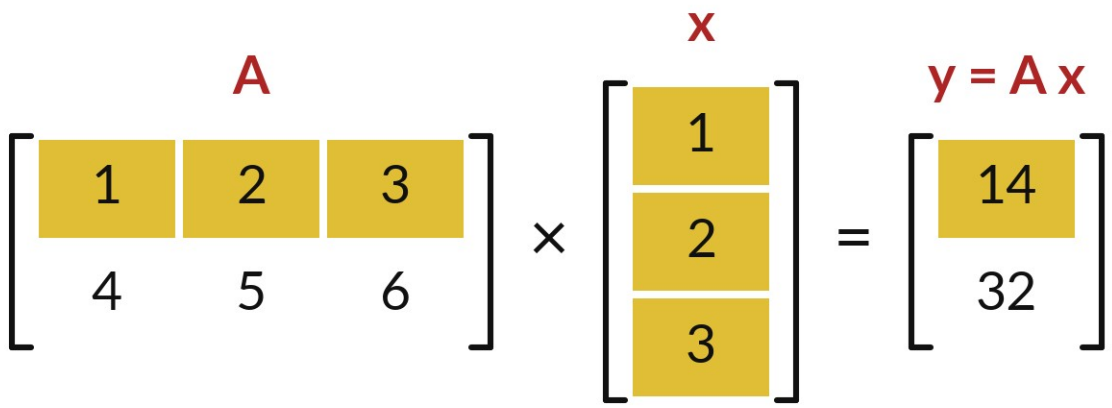

**M has rows (1, 1) and (0, 1). Red arrow (1, 0) lands on (1, 0). Black arrow (0, 1) lands on (1, 1). These two are the columns of M.**

Every other point follows: the grid tilts.

Corner (1, 3) lands on (4, 3). The next two slides show the rule and the working.

To read a transformation, read the columns.

# Matrix times vector: each entry of the answer is a dot product of one row with the vector




The gold row of A, times x, gives the gold entry of the answer

**1 × 1 + 2 × 2 + 3 × 3 = 14**

**Multiply A (2 × 3) by x (3 entries). Dot each row of A with x in turn.**

Row 1: 1 × 1 + 2 × 2 + 3 × 3 = 14, the gold entry.

Row 2: 4 × 1 + 5 × 2 + 6 × 3 = 32.

The answer y = (14, 32) has one entry per row of A.

# Matrix times vector as a formula, and the same calculation for a point of the shape

Matrix times vector

$$y_i = \sum_{j=1}^{n} A_{ij} x_j$$

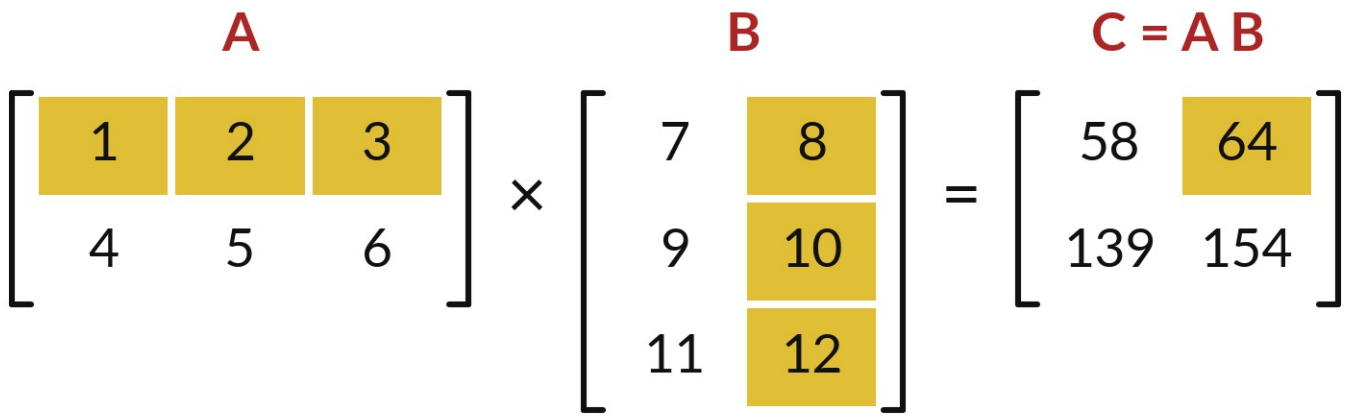
Shear, corner (1, 3)

$$1 \times 1 + 1 \times 3 = 4, \qquad 0 \times 1 + 1 \times 3 = 3$$

| Symbol | What it means | In our data |
|---|---|---|
| **$A_{ij}$** | Entry in row i, column j of A. | $A_{12} = 2$ |
| **$x_j$** | Entry j of the vector x. It meets column j of A. | $x_2 = 2$ |
| **$y_i$** | Entry i of the answer: the dot product of row i of A with x. | $y_1 = 14$ |
| **Σ, j, n** | Add up over j = 1 to n, the columns of A. | n = 3 |
| **Shear on (1, 3)** | Row 1 of M with (1, 3), then row 2 of M with (1, 3). | The point (1, 3) lands on (4, 3) |

Python: A @ x. Spreadsheet: =MMULT(A, x) as an array formula.

# Matrix times matrix: each entry of the answer is a dot product of a row of A with a column of B




Gold row of A, gold column of B: together they give the gold entry of C

$$1 \times 8 + 2 \times 10 + 3 \times 12 = 64$$

**Multiply A (2 × 3) by B (3 × 2). Entry (i, k) of the answer is row i of A dotted with column k of B.**

Gold: row 1 of A, column 2 of B: 8 + 20 + 36 = 64.

C = A B has rows (58, 64) and (139, 154).

C has the rows of A and the columns of B: 2 × 2.

# The matrix product as a formula, with a count of the multiplications

Matrix product

$$C_{ik} = \sum_{j=1}^{n} A_{ij} B_{jk}$$

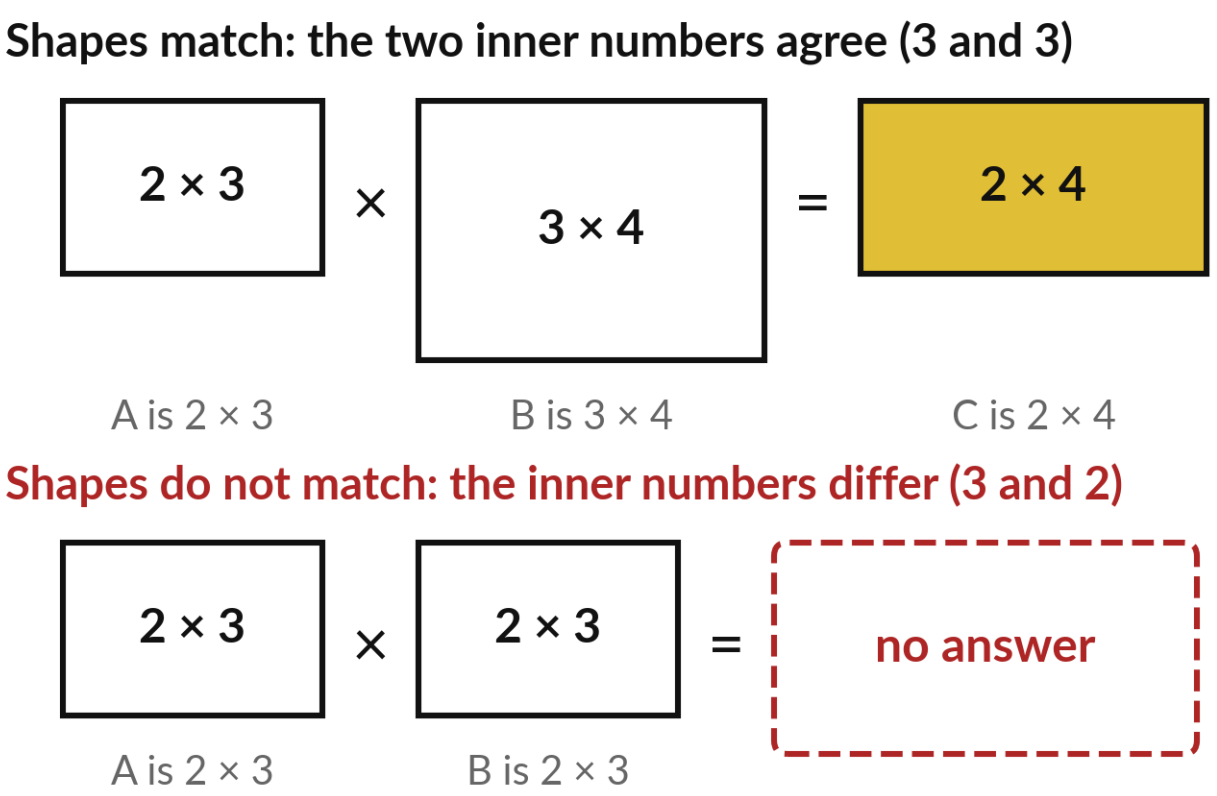
Our entry

$$C_{12} = 1 \times 8 + 2 \times 10 + 3 \times 12 = 64$$

| Symbol | What it means | In our data |
|---|---|---|
| **A, B, C** | A is m × n, B is n × p. Then C = A B is m × p. | C is 2 × 2 |
| **Σ, j, n** | Add up over j = 1 to n. | j = 1, 2, 3 |
| **$C_{ik}$** | Row i of A dotted with column k of B. | $C_{12}$ = 64 |
| **Cost** | m × p entries, n multiplications each. | 2 × 3 × 2 = 12 |

Python: A @ B. Spreadsheet: =MMULT(A, B) over a 2 × 2 range, entered as an array formula.

# Shapes must match: inner numbers agree, outer numbers give the answer's shape




**A (m × n) times B (n × p) works when the columns of A equal the rows of B. The answer is m × p.**

2 × 3 times 3 × 4 works: inner 3 and 3. The answer is 2 × 4.

2 × 3 times 2 × 3 fails: a row has 3 entries, a column 2.

Check the shapes first.

# The order matters: A B and B A are different matrices

## A B

A 2 × 3, B 3 × 2.

A B is 2 × 2, rows (58, 64) and (139, 154).

Works: inner 3 and 3.

## B A

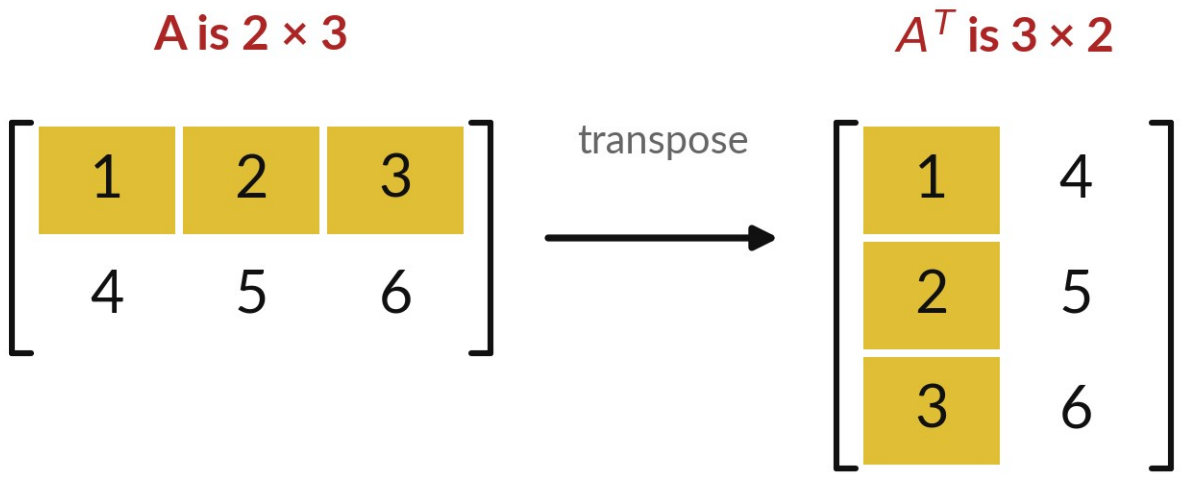
B 3 × 2, A 2 × 3.

B A is 3 × 3, first row (39, 54, 69).

Works: inner 2 and 2.

**The order matters: A B is usually different from B A.**

# The transpose flips rows into columns




**Rows become columns. Entry (i, j) of A is entry (j, i) of the transpose.**

**Transpose: write each row of A as a column. The result is written $A^T$, with a small T.**

A is 2 × 3, so $A^T$ is 3 × 2.

Entry (i, j) of A becomes entry (j, i).

Dot product: $u \cdot v$ equals $u^T v$.

# The dot product is a matrix product of a row and a column

Dot product as a matrix product

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v}$$

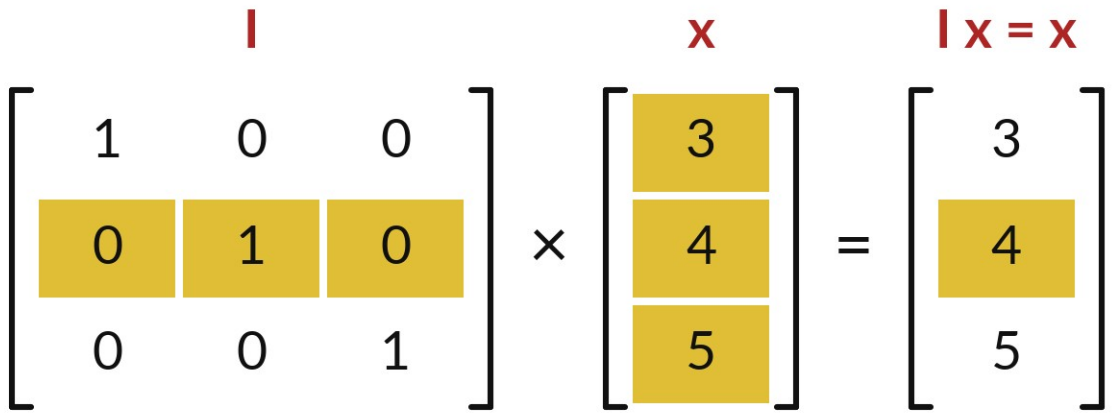
Our numbers

$$(1,\ 2,\ 3) \cdot (4,\ 0,\ 2) = 1 \times 4 + 2 \times 0 + 3 \times 2 = 10$$

| Symbol | What it means | In our data |
|---|---|---|
| $\mathbf{u}^T$ | The transpose of u: a row of 3 numbers, a 1 × 3 matrix. | (1, 2, 3) |
| $\mathbf{v}$ | A column of 3 numbers, a 3 × 1 matrix. | (4, 0, 2) |
| $\mathbf{u}^T \mathbf{v}$ | Shapes: 1 × 3 times 3 × 1. The inner numbers match and the answer is 1 × 1: one number. | 10 |

From now on a vector stands as a column; on paper we still write (1, 2, 3) across to save space. A row is written $u^T$.

# The identity matrix leaves every vector unchanged




The gold row of I picks the gold entry of x

$$0 \times 3 + 1 \times 4 + 0 \times 5 = 4$$

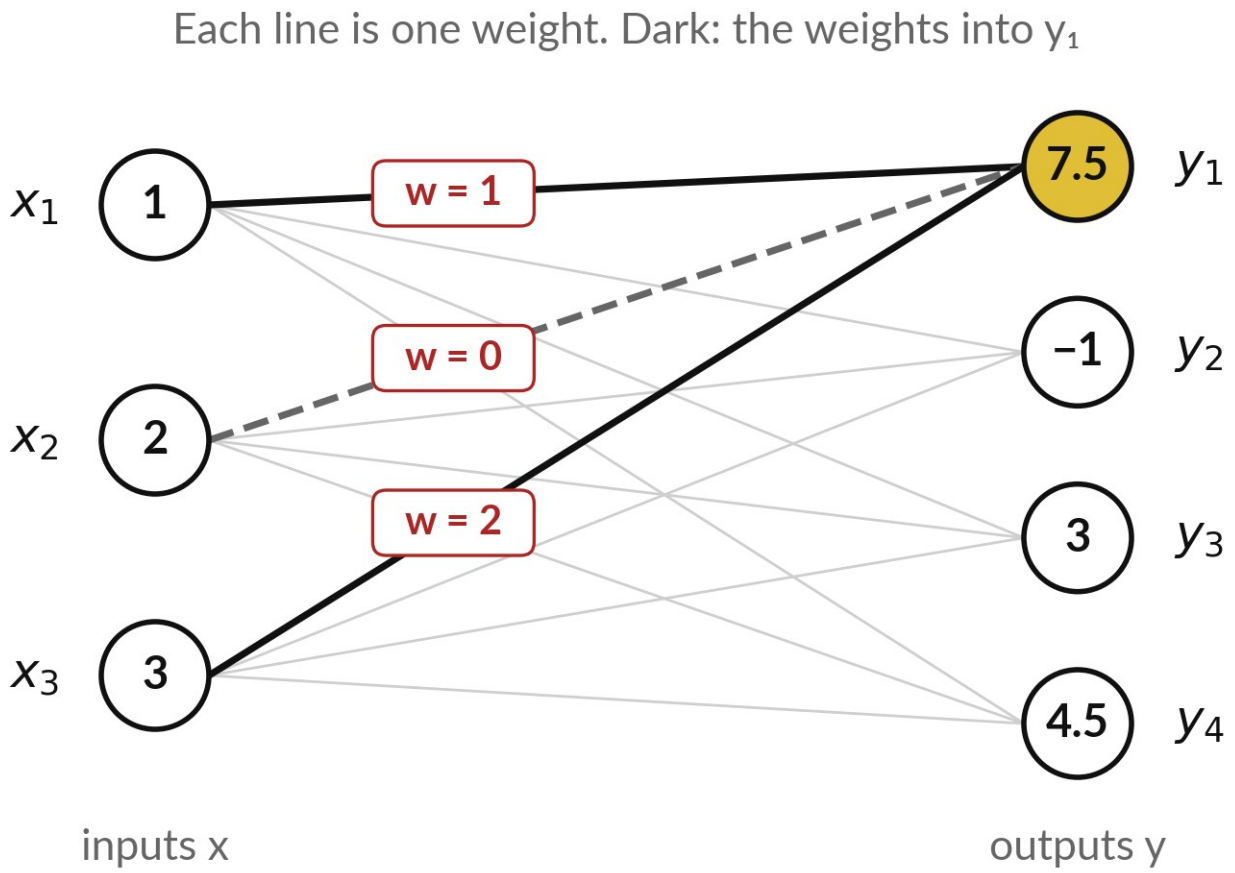
**Identity matrix I: 1s on the diagonal, 0s everywhere else. For any vector x, I x = x.**

Each row of I picks one entry of x. Row 2 picks 4.

It is the matrix version of multiplying by 1.

Session 4 uses it to define the inverse.

# A linear layer computes y = W x + b; real layers then apply an activation function




**A neural network is a chain of layers. This layer turns 3 inputs x into 4 outputs y. W is the weight matrix (4 × 3), b the bias vector.**

Illustrative: x = (1, 2, 3).

Output 1 (dark lines): 1 × 1 + 0 × 2 + 2 × 3 = 7, plus bias 0.5 gives 7.5. The weight from $x_2$ is 0 (dashed).

Training sets W and b: the parameters.

# The layer y = W x + b, with every symbol named and worked in full

One linear layer

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}$$

Our numbers

$$\mathbf{y} = (7.5,\ -1,\ 3,\ 4.5)$$

| Symbol | What it means | In our data |
|---|---|---|
| **x** | The input vector, 3 entries. | (1, 2, 3) |
| **W** | The weight matrix, 4 × 3: 4 outputs, 3 inputs. | rows (1, 0, 2), (0, 1, −1), (2, 1, 0), (−1, 1, 1) |
| **W x** | Four dot products: 7, −1, 4 and 4. | (7, −1, 4, 4) |
| **b** | The bias vector. It is added entry by entry. | (0.5, 0, −1, 0.5) |
| **y** | The output: W x + b. | (7.5, −1, 3, 4.5) |

A real layer then applies an activation function to each output (Session 11). Python: W @ x + b.

# Layer 2 takes the 4 outputs of layer 1 as its inputs

Layer 2

$$\mathbf{y}_2 = W_2\,\mathbf{y}_1 + \mathbf{b}_2$$

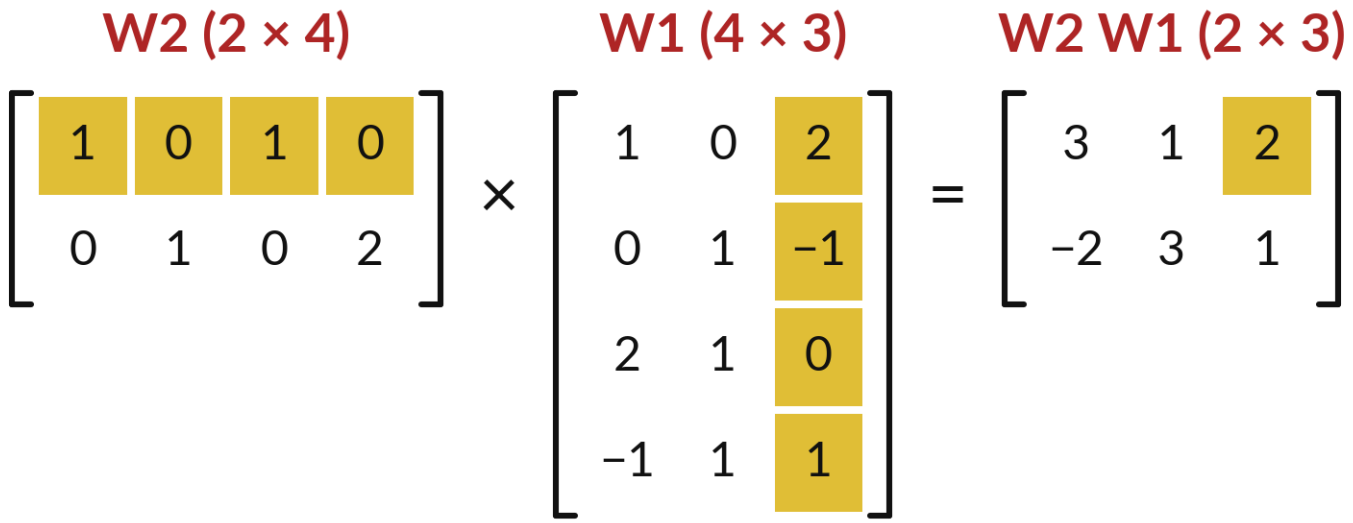
Our numbers

$$W_2\,\mathbf{y}_1 = (10.5,\ 8), \qquad \mathbf{y}_2 = (10.5,\ 9)$$

| Symbol | What it means | In our data |
|---|---|---|
| $\mathbf{y_1}$ | The output of layer 1, which is the input of layer 2. | (7.5, −1, 3, 4.5) |
| $\mathbf{W_2}$ | The weight matrix of layer 2, 2 × 4. | rows (1, 0, 1, 0) and (0, 1, 0, 2) |
| $\mathbf{b_2}$ | The bias vector of layer 2, 2 entries. | (0, 1) |
| $\mathbf{y_2}$ | The output of layer 2: $W_2\ y_1 + b_2$. | (10.5, 9) |

Activation: a nonlinear function (graph not a straight line, e.g. max(0, x)) on each output (Session 11).

# Stacking two layers is a matrix times a matrix




Gold row of W2 with gold column of W1 gives the gold entry

$$1 \times 2 + 0 \times (-1) + 1 \times 0 + 0 \times 1 = 2$$

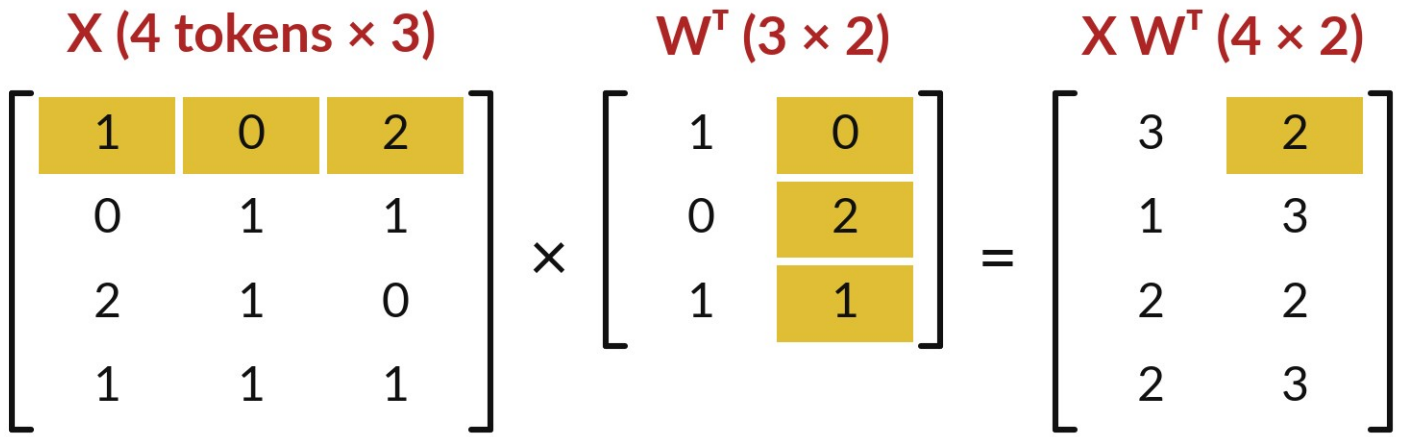
**With no biases, layer 2 after layer 1 is W2 (W1 x) = (W2 W1) x. Multiply the weights first.**

W2 W1 is 2 × 3: rows (3, 1, 2) and (−2, 3, 1).

On x = (1, 2, 3) it gives (11, 7), as two layers in turn do.

With no activation function (nonlinear step), two layers collapse into one.

# A whole sentence goes through a layer at once: tokens as rows, so the layer is X $W^T$

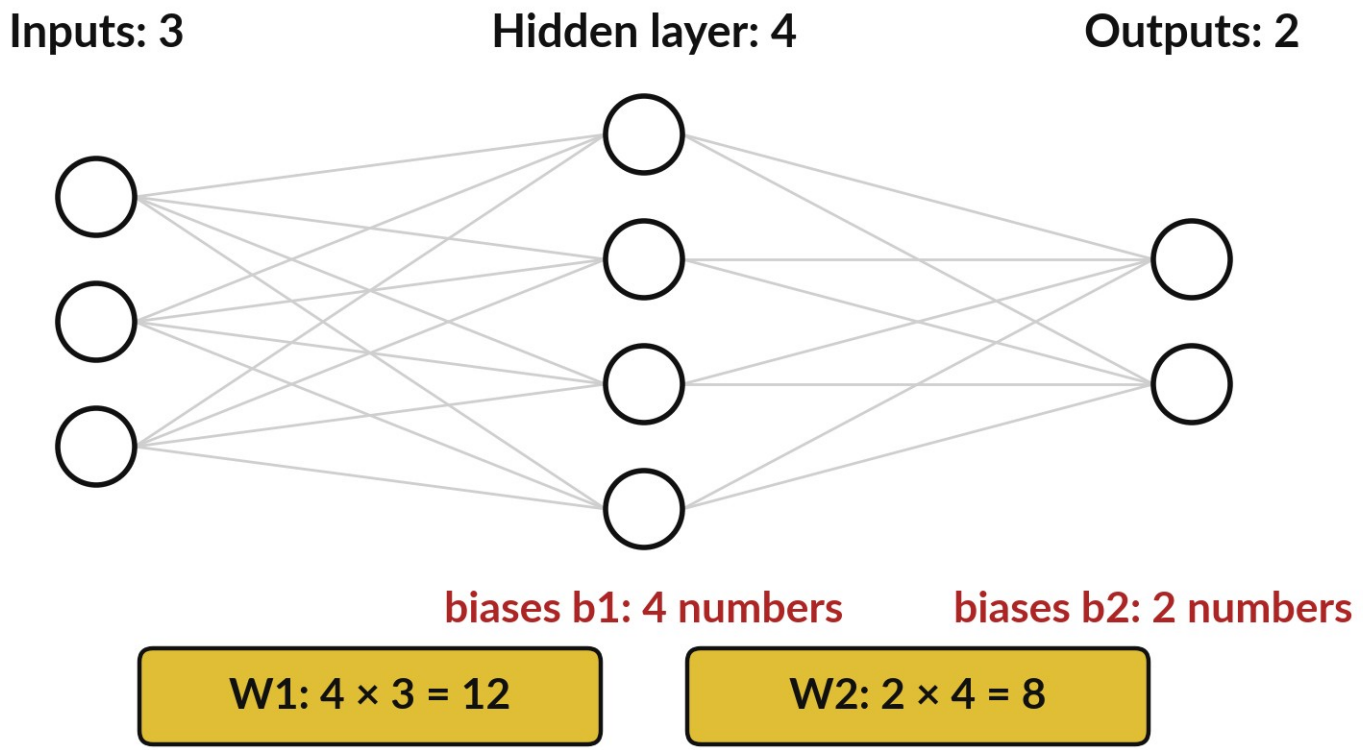



Gold row of X (token 1), gold column of $W^T$ (row 2 of W): the gold entry

$$1 \times 0 + 0 \times 2 + 2 \times 1 = 2$$

Illustrative. W is 2 outputs × 3 inputs. Tokens as rows: X $W^T$ (as in PyTorch).

# Counting the numbers in the matrices gives the parameters of a small network




**Parameters: the numbers a model learns. Toy network: 3 inputs, a hidden layer of 4 (between input and output), 2 outputs.**

Layer 1: W1 is 4 × 3 = 12 weights, plus 4 biases.

Layer 2: W2 is 2 × 4 = 8 weights, plus 2 biases.

Total: 12 + 4 + 8 + 2 = 26 parameters.

# Parameters of a layer: m × n weights plus m biases

Parameters of one layer

$$\text{parameters} = m \times n + m$$

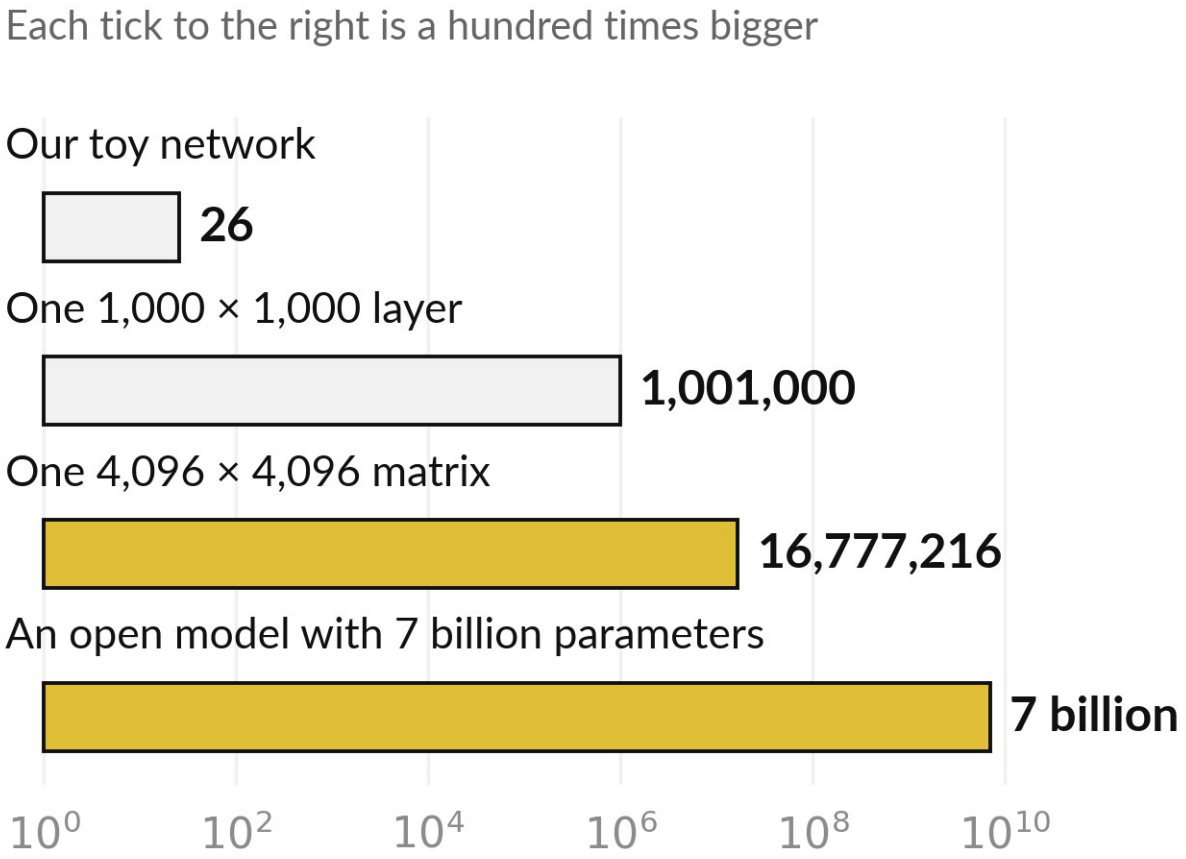
Our network

$$(4 \times 3 + 4) + (2 \times 4 + 2) = 16 + 10 = 26$$

| Symbol | What it means | In our data |
|---|---|---|
| **n** | Number of inputs of the layer, which is the number of columns of W. | layer 1: n = 3 |
| **m** | Number of outputs, which is the number of rows of W and the length of b. | layer 1: m = 4 |
| **m × n** | The weights in W. | 12 |
| **+ m** | The biases in b. | 4 |
| **Bigger** | A layer from 1,000 inputs to 1,000 outputs. | 1,000 × 1,000 + 1,000 = 1,001,000 |

Python: W.size + b.size. Spreadsheet: ROWS(W) * COLUMNS(W) + ROWS(b).

# Billions of parameters means billions of numbers in matrices like these




**Open models publish their size: Llama 2 has 7, 13 and 70 billion parameters (Touvron et al., 2023).**

Logarithmic scale, as in Session 1.

A byte is 8 bits. A 16-bit number takes 2 bytes, so 7 billion numbers fill 14 gigabytes.

Training sets every one.

# Common mistake: multiplying two matrices entry by entry instead of row by column

## Wrong

Multiply matching entries of A and B.

Python: A * B, entry by entry. For our 2 × 3 and 3 × 2 it fails.

## Right

Dot products of rows of A with columns of B.

Python: A @ B. Spreadsheet: MMULT.

Check the shapes first.

**The star (*) multiplies entry by entry. The at sign (@) gives the matrix product.**

# Read the recipe before you run it: matrices in NumPy

```
A = np.array([[1, 2, 3], [4, 5, 6]])
x = np.array([1, 2, 3])
y = A @ x
C = A @ B
At = A.T
I = np.eye(3)
z = W @ x + b
Z = X @ W.T + b
n = W.size + b.size
```

## In plain words

Line 1: a matrix from a list of rows. A.shape gives (2, 3).

Lines 3 and 4: the at sign is the matrix product. Spreadsheet: MMULT.

Line 5: the transpose. Spreadsheet: TRANSPOSE.

Line 6: the identity matrix of size 3 × 3.

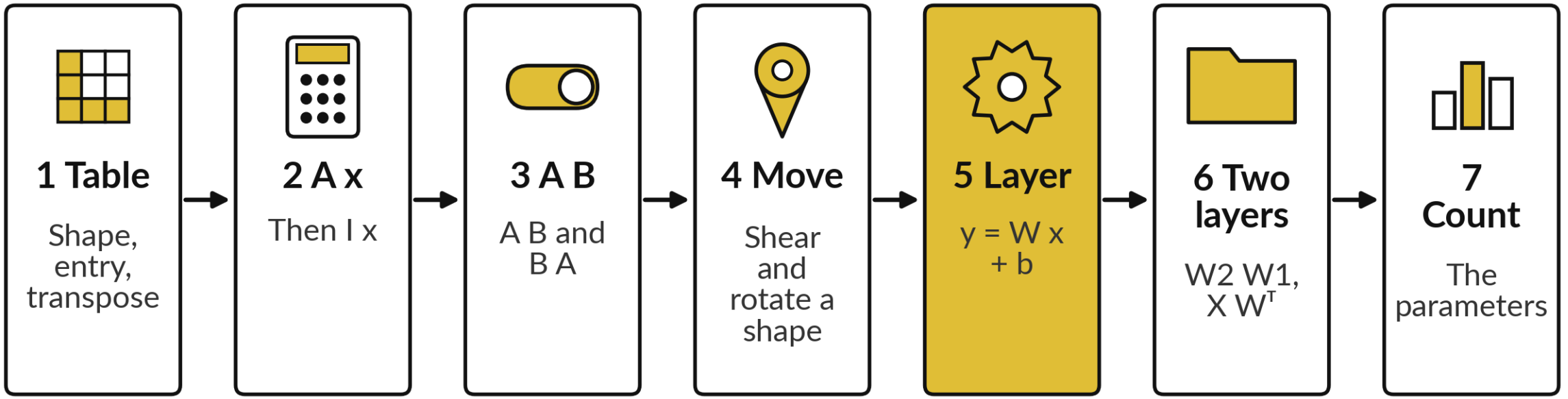
Lines 7 and 8: one layer, for one column x, then for one row per input. Line 9: the parameters.



# Part 2: the lab

Close the slides. Open the notebook or the workbook.

# Seven lab steps take you from a table to a count of parameters




Predict, run, read, change one thing. Workbook: Steps 1 to 4 on Products, 5 to 7 on Layers.

# Check your numbers before you move on

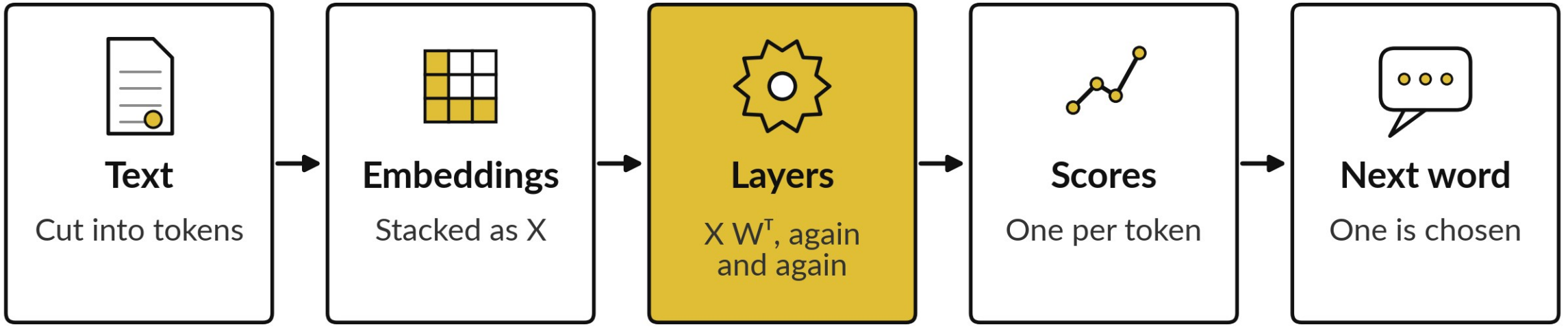
| | What you compute | You should get |
|---|---|---|
| | **Learner table: shape, and entry in row 2, column 3** | 3 × 3 and 58 |
| | **Shape of the transpose of A (2 × 3)** | 3 × 2 |
| | **A x for x = (1, 2, 3)** | (14, 32) |
| | **A B, rows 1 and 2; shape of B A** | (58, 64), (139, 154); 3 × 3 |
| | **Shear moves the corner (1, 3) to** | (4, 3) |
| | **Layer 1: y1 = W1 x + b1** | (7.5, −1, 3, 4.5) |
| | **Layer 2: W2 y1 + b2** | (10.5, 9) |
| | **Parameters: toy network; 1,000 × 1,000 layer** | 26; 1,001,000 |

If your number differs, find the step that went wrong. Do not change a number to match.

# Four messages you may meet, what they mean and what to do

| | What you see | What it means | What to do |
|---|---|---|---|
|  | **ValueError: matmul mismatch in the core dimension** | The inner numbers of the shapes differ. | Print A.shape and B.shape. They must match inside. |
|  | **A * B gives an error or the wrong table** | The star multiplies entry by entry. | Use A @ B for the matrix product. |
|  | **#VALUE! or only one cell filled in a spreadsheet** | The formula was not an array over the whole answer range. | Excel: select the gold range, type, press Ctrl Shift Enter. Google Sheets: the same keys in the first cell. |
|  | **IndexError for entry 2, 3** | Python counts from 0, so row 2 and column 3 is entry 1, 2. | Subtract 1 from the row and the column number. |

# Where this appears in AI: most of the arithmetic in each layer is matrix multiplication




Published transformer design (Vaswani et al., 2017): most arithmetic is matrix products.

# Where this appears in AI: why graphics processors (GPUs) run models

## Why matrix products suit a GPU

**Each entry is independent**
*One dot product per entry of the answer*

**Work can be shared**
*A GPU has thousands of small units working at once*

**Two 4,096 × 4,096 matrices**
*68,719,476,736 multiplications (about 69 billion)*

**Published training reports**
*Describe clusters of GPUs or similar accelerators*

**A GPU (graphics processing unit) was first made to draw images, which needs many simple products.**

m × n × p multiplications grow fast with size.

Many dot products at once suit a GPU.

Training and running large models need fast matrix products.

# Where this appears in AI: check an assistant with your numbers

8 minutes Hands-on activity

## Ask an AI assistant to multiply our two matrices.

Give it A with rows (1, 2, 3), (4, 5, 6) and B with rows (7, 8), (9, 10), (11, 12). Ask for A B.

Compare: rows (58, 64) and (139, 154).

Ask for the parameters of a 1,000 by 1,000 layer: expect 1,001,000.

**You keep**

A note: the assistant answers, your lab numbers, and whether they agree.

# Exit check: three questions, then compare with the short answers

| | Question | Short answer |
|---|---|---|
| | **Can a 3 × 4 matrix multiply a 4 × 2 matrix? If yes, what is the shape of the answer?** | Yes. The inner numbers are 4 and 4. The answer is 3 × 2. |
| | **What is (1, 2; 3, 4) times the vector (1, 1)? Write the matrix as rows (1, 2) and (3, 4).** | Row 1: 1 + 2 = 3. Row 2: 3 + 4 = 7. The answer is (3, 7). |
| | **How many parameters does a layer have with 10 inputs and 5 outputs, with biases?** | 5 × 10 + 5 = 55. |

# Save your lab file and add three rules to your sheet

## What you keep from today

Keep your saved notebook or workbook, with every check on OK. Keep the idea of the day: a layer is y = W x + b, and the parameters of a model are the numbers in its matrices and biases.

### Our question today

*A language model has billions of parameters. What are those numbers, and what does the model do with them?*

### Next session

Session 4: Solving linear systems. We write equations as a matrix, and meet the inverse, the determinant and the rank.

**Take away**

**Your lab file**
*Saved, all checks OK*

**The idea**
$y = W x + b$

**Formula sheet**
*Shape rule;* $C_i = \Sigma A_i B$; $m$ $n + m$

**Next session**
*Solving linear systems*